// Redes de Computadores II // aula 07 · 60 minutos

HTTP e HTTPS

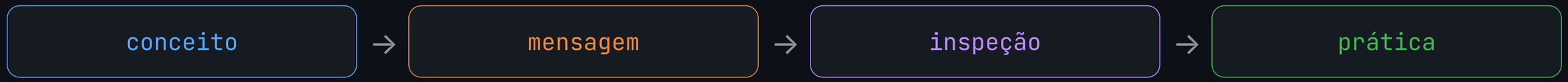
Como aplicações web solicitam, transportam e recebem informações



// objetivos

O que estudaremos hoje

- [ ] Significado e utilidade do HTTP.
- [ ] Diferença entre HTTP e HTTPS.
- [ ] Cabeçalhos e corpos.
- [ ] Inspeção no Chrome.
- [ ] Funcionamento da comunicação.
- [ ] Métodos e caminhos.
- [ ] Códigos de resposta.
- [ ] Relação com o IDS.



// pergunta inicial

O que acontece depois que digitamos uma URL?

barra de endereço

```
http://127.0.0.1:8000/index.html
```

01 O navegador envia somente esse texto?

02 Como o servidor identifica o recurso desejado?

03 Como o navegador descobre se o pedido funcionou?

04 Onde aparecem os dados enviados e recebidos?

// roteiro

Organização da aula

1. O que é HTTP
2. Como HTTP e HTTPS funcionam
3. Estrutura das requisições
4. Estrutura das respostas
5. Inspeção no navegador
6. Laboratório individual

// revisão

Cliente e servidor

CLIENTE · navegador

01 Inicia a comunicação.

02 Monta uma solicitação.

03 Envia o pedido.

04 Recebe e interpreta a resposta.

SERVIDOR · Python

01 Permanece aguardando.

02 Interpreta o pedido.

03 Executa uma operação.

04 Produz uma resposta.

// conclusão

Cliente e servidor são papéis desempenhados por processos.

// conceitos

Protocolo de comunicação

> Um protocolo é um conjunto de regras que determina como as mensagens serão formadas, enviadas e interpretadas.

01 Formato.

02 Ordem dos campos.

03 Significado dos campos.

01 Comportamento esperado.

02 Representação de erros.

> // compatibilidade
> Para se comunicarem, cliente e servidor precisam seguir regras compatíveis.

Significado de HTTP

sigla

HTTP

Hypertext Transfer Protocol

Protocolo de Transferência de Hipertexto

> HTTP é um protocolo da camada de aplicação utilizado na troca de mensagens entre clientes e servidores.

APESAR DO NOME, TRANSPORTA

HTML JSON texto imagens áudio vídeo arquivos

Utilidade do HTTP

01 Solicitar um recurso.

02 Enviar informações.

03 Atualizar dados.

04 Solicitar exclusão.

05 Receber o resultado.

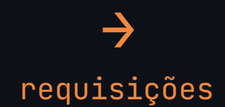
06 Transferir conteúdos diversos.

exemplos

```
Abrir uma página
Consultar produtos
Enviar um formulário
Fazer login
Publicar uma mensagem
Consumir uma API
```

Quem utiliza HTTP

- Chrome
- Firefox
- Aplicativo móvel
- curl
- Python requests
- Frontend JavaScript

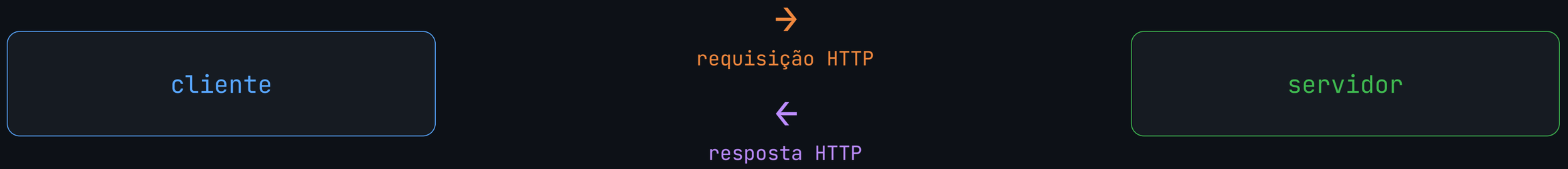


servidor HTTP

sites · APIs · microsserviços

> Navegadores, aplicativos, APIs, microsserviços, dispositivos conectados, programas de terminal e sistemas corporativos.

Modelo de requisição e resposta



01 Cliente envia uma requisição.

02 Servidor interpreta o pedido.

03 Servidor produz uma resposta.

04 Cliente interpreta a resposta.

> // regra
> A comunicação HTTP é organizada em pares de requisição e resposta.

Etapas de um acesso web

01 Usuário informa a URL.

02 Cliente interpreta o endereço.

03 Cliente localiza o destino.

04 Transporte é preparado (TCP).

01 Cliente envia a requisição HTTP.

02 Servidor interpreta o pedido.

03 Servidor envia a resposta.

04 Cliente apresenta o resultado.

> // observação
> Com um endereço IP, como `127.0.0.1`, não é preciso consultar o DNS.

HTTP não mantém automaticamente o estado da aplicação

> Cada requisição HTTP pode ser interpretada como uma solicitação independente.

HTTP NÃO GUARDA SOZINHO

01 Usuário autenticado.

02 Carrinho de compras.

03 Progresso de uma tarefa.

04 Preferências.

A APLICAÇÃO USA

01 Cookies.

02 Tokens.

03 Sessões.

04 Bancos de dados.

> // resumo
> O protocolo transporta mensagens. A aplicação decide como manter seu estado.

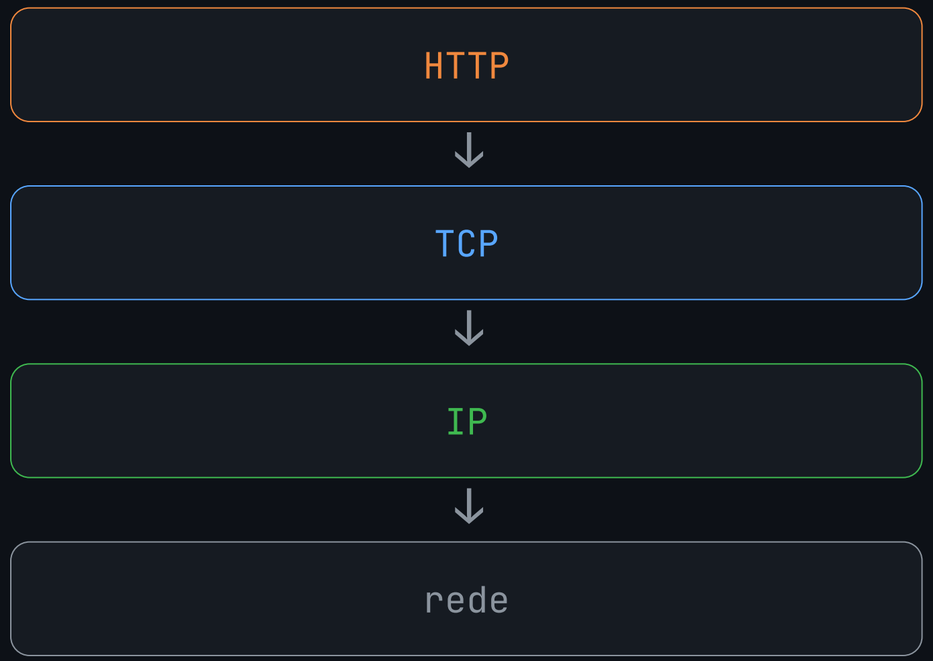
Versões do HTTP

HTTP/1.1	HTTP/2	HTTP/3
Mensagens textuais	Quadros binários	Usa QUIC
Normalmente sobre TCP	Normalmente sobre TCP	QUIC funciona sobre UDP
Foco desta aula	Várias solicitações na mesma conexão	Mantém a semântica HTTP

```
// o que permanece
```

Métodos, caminhos e códigos continuam importantes em todas as versões.

HTTP, TCP e IP possuem funções diferentes



HTTP → define pedidos e respostas da aplicação.

TCP → transporta um fluxo confiável de bytes.

IP → encaminha os dados entre endereços.

```
// nesta aula
```

Observaremos HTTP/1.1 funcionando sobre TCP.

// https

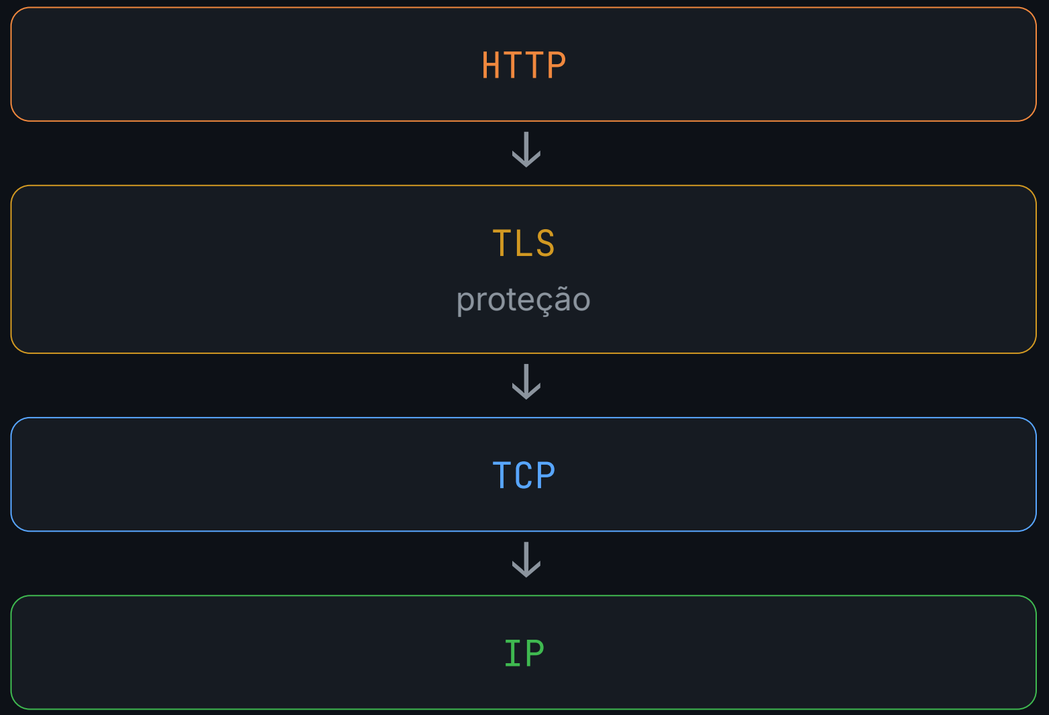
Significado de HTTPS

sigla

HTTPS

Hypertext Transfer Protocol Secure

> HTTPS é a comunicação HTTP protegida pelo protocolo TLS.



Proteções oferecidas pelo TLS

01 Confidencialidade — o conteúdo fica criptografado durante o transporte.

02 Integridade — alterações indevidas podem ser detectadas.

03 Autenticação do servidor — o certificado ajuda a verificar a identidade do servidor.

> // observação
> TLS também pode autenticar clientes, mas isso é menos comum em aplicações web convencionais.

HTTP e HTTPS

HTTP	HTTPS
HTTP sem proteção TLS	HTTP protegido por TLS
Esquema `http://`	Esquema `https://`
Porta padrão `80`	Porta padrão `443`
Conteúdo legível se interceptado	Conteúdo criptografado no transporte
Sem certificado TLS	Certificado autentica o servidor

> Os dois usam requisições e respostas HTTP. HTTPS acrescenta a proteção do TLS.

O que HTTPS protege

PROTEGIDO

01 Método.

02 Caminho.

03 Cabeçalhos HTTP.

04 Corpo da requisição.

05 Código e conteúdo da resposta.

AINDA OBSERVÁVEL

01 Endereços IP.

02 Portas.

03 Horários.

04 Duração.

05 Quantidade aproximada de dados.

> // limite
> HTTPS protege o conteúdo em trânsito, mas não esconde que a comunicação existe.

// erro comum

HTTPS não garante que a aplicação seja segura

HTTPS protege a comunicação durante o transporte. Ele não impede automaticamente:

- [ ] Falhas de programação.
- [ ] Senhas fracas.
- [ ] Páginas maliciosas.
- [ ] Permissões incorretas.
- [ ] Ataques contra a aplicação.
- [ ] Vazamentos no servidor.

// conclusão

Uma aplicação pode usar HTTPS e ainda ter vulnerabilidades.

// url

Estrutura de uma URL

URL

```
http://127.0.0.1:8000/produtos/10?detalhes=true#avaliacoes
```

PARTE	FUNÇÃO
http	esquema
127.0.0.1	endereço do destino
8000	porta
/produtos/10	caminho
?detalhes=true	parâmetros de consulta
#avaliacoes	fragmento

> // fragmento
> O fragmento é usado pelo cliente e não vai na requisição enviada ao servidor.

Endereço, porta e caminho

endereço IP

Ajuda a localizar a interface de destino.

`127.0.0.1`

porta

Direciona a comunicação ao serviço.

`8000`

caminho

Identifica o recurso dentro do serviço.

`/produtos/10`

juntos

```
127.0.0.1:8000/produtos/10
```

// requisição

Partes de uma requisição HTTP

requisição HTTP/1.1 — cliente → servidor

```
GET /index.html HTTP/1.1
Host: 127.0.0.1:8000
Accept: text/html
```

01 Linha da requisição.

02 Cabeçalhos.

03 Linha vazia.

04 Corpo opcional.

// linha vazia

A linha vazia indica o final dos cabeçalhos.

Linha da requisição

linha inicial

```
GET  /index.html  HTTP/1.1
```

CAMPO	PAPEL
`GET`	método
`/index.html`	caminho
`HTTP/1.1`	versão

> A linha inicial informa a ação desejada, o recurso solicitado e a versão do protocolo.

Métodos HTTP

MÉTODO	INTENÇÃO
`GET`	Solicitar um recurso
`POST`	Enviar dados para processamento
`PUT`	Criar ou substituir um recurso
`PATCH`	Alterar parte de um recurso
`DELETE`	Solicitar a remoção de um recurso
`HEAD`	Solicitar somente os cabeçalhos
`OPTIONS`	Consultar opções de comunicação

> `// importante`
> O servidor implementa o comportamento associado a cada método.

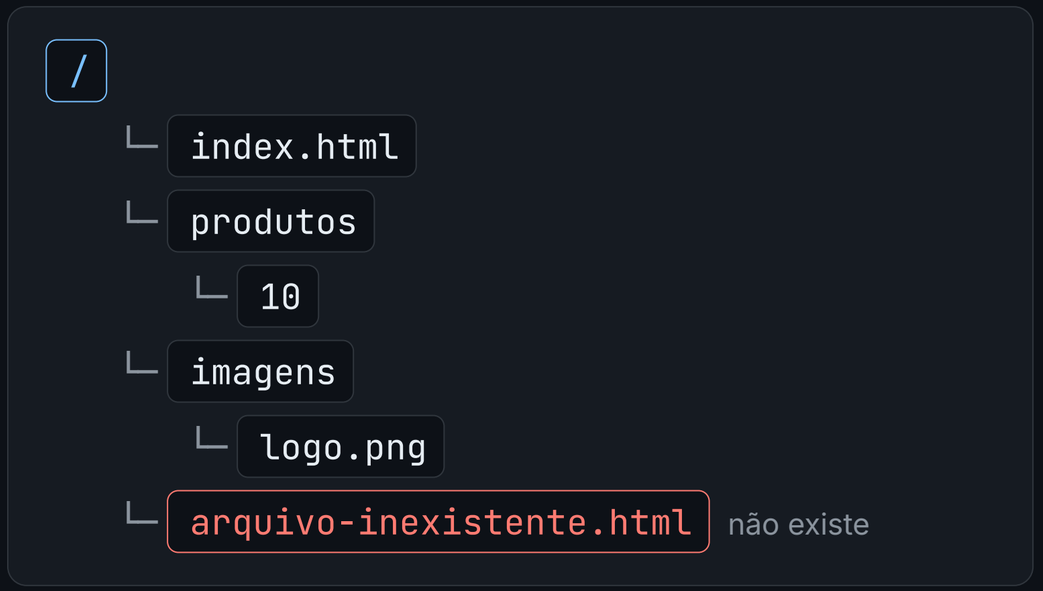
GET e POST

GET	POST
Solicita uma representação	Envia dados para processamento
Sem corpo nesta aula	Frequentemente usa corpo
Pode usar parâmetros na URL	Pode transportar JSON ou formulário
Ex.: consultar produto	Ex.: cadastrar mensagem

```
GET /produtos/10 HTTP/1.1
```

```
POST /mensagens HTTP/1.1
```

Caminho do recurso



> O caminho informa o recurso ou a operação desejada dentro do serviço.

01 Conteúdos diferentes.

02 Códigos diferentes.

03 Comportamentos diferentes.

Parâmetros de consulta

caminho com consulta

```
/produtos?categoria=livros&pagina=2
```

TRECHO	PAPEL
`/produtos`	caminho
`categoria=livros`	1º parâmetro
`pagina=2`	2º parâmetro

01 `?` inicia a consulta.

02 `&` separa parâmetros.

03 `=` liga nome e valor.

> // não confunda
> Caminho e parâmetros cumprem funções diferentes.

Cabeçalhos da requisição

cabeçalhos

```
Host: 127.0.0.1:8000
User-Agent: Mozilla/5.0
Accept: text/html
Accept-Language: pt-BR
Content-Type: application/json
```

`Host` → serviço de destino.

`User-Agent` → cliente declarado.

`Accept` → formatos aceitos.

`Accept-Language` → idioma.

`Content-Type` → formato do corpo.

```
// formato
```

Um cabeçalho é formado por nome e valor.

Accept e Content-Type

ACCEPT

Formato que o cliente aceita receber.

```
Accept: application/json
```

CONTENT-TYPE

Formato do corpo que está sendo enviado.

```
Content-Type: application/json
```

```
// conclusão
```

Um descreve a preferência para a resposta; o outro descreve o conteúdo da mensagem atual.

Corpo da requisição

requisição com corpo — cliente → servidor

```
POST /mensagens HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"nome":"Ana","mensagem":"Olá"}
```

01 O corpo vem depois da linha vazia.

02 Pode transportar dados.

03 JSON é comum em APIs.

04 Nem toda requisição tem corpo.

// análise

Requisição HTTP anotada

requisição

```
POST /mensagens HTTP/1.1
Host: 127.0.0.1:8000
Accept: application/json
Content-Type: application/json

{"texto":"Olá, servidor!"}
```

TRECHO	FUNÇÃO
`POST`	método
`/mensagens`	caminho
`HTTP/1.1`	versão
`Host`	destino lógico
`Accept`	resposta desejada
`Content-Type`	formato do corpo
`{"texto":...}`	corpo

// resposta

Partes de uma resposta HTTP

resposta HTTP/1.1 — servidor → cliente

```
HTTP/1.1 200 OK
Content-Type: text/html

<h1>Servidor funcionando</h1>
```

01 Linha de status.

02 Cabeçalhos.

03 Linha vazia.

04 Corpo opcional.

Linha de status

linha inicial

```
HTTP/1.1  200  OK
```

CAMPO	PAPEL
HTTP/1.1	versão
200	código
OK	descrição

> A linha de status resume o resultado da solicitação.

Famílias de códigos HTTP

1xx	2xx	3xx	4xx	5xx
100–199	200–299	300–399	400–499	500–599
Informação	Sucesso	Redirecionamento	A solicitação não pôde ser atendida	O servidor não conseguiu processar

```
// leitura rápida
```

O primeiro dígito já diz a família do resultado.

https://pt.wikipedia.org/wiki/Lista_de_c%C3%B3digos_de_estado_HTTP

https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Reference/Status

Códigos HTTP frequentes

CÓDIGO	SIGNIFICADO
200 OK	Solicitação processada
201 Created	Recurso criado
204 No Content	Sucesso sem corpo
301 Moved Permanently	Recurso movido
400 Bad Request	Solicitação inválida
401 Unauthorized	Autenticação necessária

CÓDIGO	SIGNIFICADO
403 Forbidden	Acesso recusado
404 Not Found	Não encontrado
405 Method Not Allowed	Método não permitido
500 Internal Server Error	Falha interna
501 Not Implemented	Não implementado
503 Service Unavailable	Indisponível

Cabeçalhos e corpo da resposta

resposta — servidor → cliente

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 23

{"status":"ok","id":10}
```

01 `Content-Type` descreve o formato.

02 `Content-Length` informa o tamanho em bytes.

03 A linha vazia encerra os cabeçalhos.

04 O JSON é o corpo.

// análise

Resposta HTTP anotada

resposta

```
HTTP/1.1  404 Not Found
Content-Type: text/html
Content-Length: 148

<h1>Arquivo não encontrado</h1>
```

TRECHO	FUNÇÃO
`HTTP/1.1`	versão
`404`	código
`Not Found`	descrição
`Content-Type`	formato do corpo
`Content-Length`	tamanho do corpo
`<h1>...</h1>`	conteúdo enviado

Respostas 200 e 404

200 OK	404 NOT FOUND
A requisição chegou ao servidor	A requisição chegou ao servidor
O servidor encontrou o recurso	O servidor não encontrou o recurso
Existe uma resposta HTTP	Existe uma resposta HTTP
O corpo contém o recurso	O corpo pode conter uma página de erro

```
// importante
```

Receber 404 comprova que o servidor recebeu e interpretou a requisição.

// erros

Código HTTP e falha de conexão

RESPOSTA HTTP

códigos

```
200
404
500
501
```

O cliente se comunicou com o servidor e recebeu uma resposta.

FALHA ANTERIOR AO HTTP

navegador

```
Conexão recusada
Tempo esgotado
Servidor inexistente
Porta incorreta
```

O cliente não recebeu uma resposta HTTP válida.

// regra

Um código de erro HTTP ainda é uma resposta do servidor.

// leitura orientada

Comunicação HTTP completa

CLIENTE

requisição →

```
GET /index.html
Host: 127.0.0.1:8000
```

→

servidor procura o recurso

←

RESPOSTA

resposta ←

```
HTTP/1.1 200 OK
Content-Type: text/html
```

O cliente recebe o HTML e apresenta a página.

// exercício guiado

Identificação dos campos

requisição

```
GET /produtos?categoria=livros HTTP/1.1
Host: loja.exemplo
Accept: application/json
```

01 Qual é o método?

02 Qual é o caminho?

03 Qual é o parâmetro?

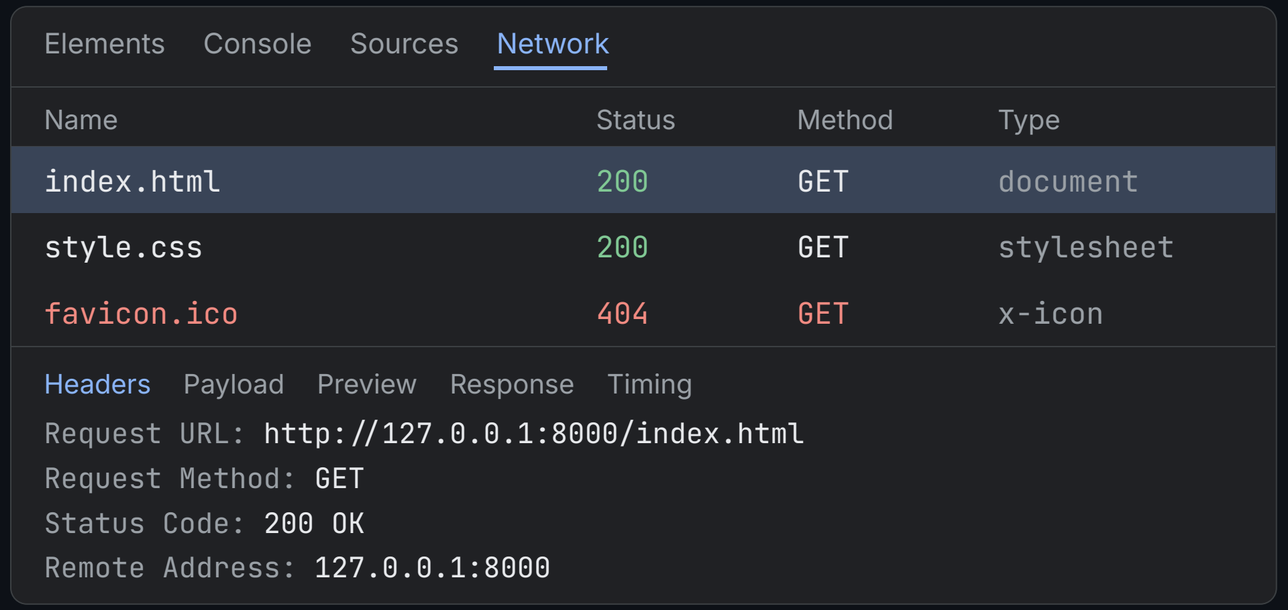
04 Qual formato o cliente aceita?

05 Existe corpo?

06 Qual é o destino informado?

// devtools

Chrome DevTools · Network



NA ABA NETWORK

01 URL e método.

02 Código e endereço remoto.

03 Cabeçalhos.

04 Corpo enviado (Payload).

05 Resposta (Response).

06 Tempo (Timing).

// demonstração

Servidor HTTP local

PowerShell · pasta aula07

```
PS> py -m http.server 8000 --bind 127.0.0.1
Serving HTTP on 127.0.0.1 port 8000 (http://127.0.0.1:8000/) ...
```

TRECHO	FUNÇÃO
`py`	executa o Python
`-m http.server`	inicia o módulo de servidor HTTP
`8000`	define a porta
`--bind 127.0.0.1`	limita o acesso ao próprio computador

barra de endereço

http://127.0.0.1:8000/

// demonstração

Comparação prática de respostas

TESTE 1 · RECURSO EXISTENTE

barra de endereço

127.0.0.1:8000/index.html

Elements Console Sources Network

Name	Status	Method	Type
index.html	200	GET	document

esperado

```
GET /index.html
200 OK
Content-Type: text/html
```

TESTE 2 · RECURSO INEXISTENTE

barra de endereço

127.0.0.1:8000/arquivo-inexistente.html

Elements Console Sources Network

Name	Status	Method	Type
arquivo-inexistente.html	404	GET	document

esperado

```
GET /arquivo-inexistente.html
404 Not Found
```

// atividade individual · 8 min + síntese 2 min

Investigação individual de uma comunicação HTTP

01 Iniciar o servidor local.

02 Abrir o DevTools.

03 Acessar `index.html`.

04 Acessar um recurso inexistente.

05 Selecionar as duas requisições.

06 Identificar os campos e comparar.

CAMPO	200	404
Método		
Caminho		
2 cabeçalhos da requisição		
Código da resposta		
2 cabeçalhos da resposta		
Tipo do corpo		
Função de cada campo		

// IDS

Métodos, caminhos, códigos, tamanhos, frequência, erros repetidos e origem/destino poderão ser monitorados.

// fechamento

Primeiro interpretamos as mensagens manualmente. Depois, criaremos programas para coletar, organizar e analisar essas informações.

// quiz · 01/15

O que significa a sigla HTTP?

A Hypertext Transfer Protocol

B High Transfer Text Protocol

C Hypertext Transport Program

D Host Transfer Protocol

// quiz · 01/15 · resposta

O que significa a sigla HTTP?

- ✓ Hypertext Transfer Protocol
- B High Transfer Text Protocol
- C Hypertext Transport Program
- D Host Transfer Protocol

// por quê

HTTP = Hypertext Transfer Protocol, o Protocolo de Transferência de Hipertexto.

Quem inicia uma comunicação HTTP?

A. O servidor

B. O cliente

C. O roteador

D. O servidor DNS

// quiz · 02/15 · resposta

Quem inicia uma comunicação HTTP?

A O servidor

✓ O cliente

C O roteador

D O servidor DNS

// por quê

O cliente envia a requisição; o servidor aguarda e responde.

// quiz · 03/15

O que é HTTPS?

A. Um protocolo sem relação com o HTTP

B. HTTP protegido por TLS

C. HTTP transportado sempre por UDP

D. HTTP sem cabeçalhos

// quiz · 03/15 · resposta

O que é HTTPS?

A. Um protocolo sem relação com o HTTP

✓ HTTP protegido por TLS

C. HTTP transportado sempre por UDP

D. HTTP sem cabeçalhos

// por quê

HTTPS é a mesma comunicação HTTP, protegida pelo TLS durante o transporte.

Qual é a porta padrão do HTTPS?

A 80

B 22

C 8000

D 443

// quiz · 04/15 · resposta

Qual é a porta padrão do HTTPS?

A 80

B 22

C 8000

✓ 443

// por quê

HTTP usa 80 e HTTPS usa 443. A 8000 foi uma escolha didática do laboratório.

// quiz · 05/15

Nesta linha de requisição, qual é o caminho?

linha da requisição

```
GET /index.html HTTP/1.1
```

A GET

B /index.html

C HTTP/1.1

D 127.0.0.1

// quiz · 05/15 · resposta

Nesta linha de requisição, qual é o caminho?

linha da requisição

```
GET /index.html HTTP/1.1
```

A GET

✓ /index.html

C HTTP/1.1

D 127.0.0.1

// por quê

GET é o método, /index.html é o caminho e HTTP/1.1 é a versão.

O que indica o final dos cabeçalhos de uma mensagem HTTP/1.1?

A. Uma linha vazia

B. O cabeçalho Content-Length

C. Um ponto e vírgula

D. O código de status

// quiz · 06/15 · resposta

O que indica o final dos cabeçalhos de uma mensagem HTTP/1.1?

✓ Uma linha vazia

B O cabeçalho Content-Length

C Um ponto e vírgula

D O código de status

// por quê

A linha vazia separa os cabeçalhos do corpo.

Qual cabeçalho informa o formato do corpo que está sendo enviado?

A Accept

B Host

C User-Agent

D Content-Type

// quiz · 07/15 · resposta

Qual cabeçalho informa o formato do corpo que está sendo enviado?

A Accept

B Host

C User-Agent

✓ Content-Type

// por quê

Content-Type descreve o corpo atual; Accept indica o que o cliente aceita receber.

// quiz · 08/15

Neste caminho, o que é categoria=livros?

caminho

```
/produtos?categoria=livros
```

A O caminho

B Um parâmetro de consulta

C Um fragmento

D Um cabeçalho

// quiz · 08/15 · resposta

Neste caminho, o que é categoria=livros?

caminho

```
/produtos?categoria=livros
```

A O caminho

✓ Um parâmetro de consulta

C Um fragmento

D Um cabeçalho

// por quê

O ? inicia a consulta; categoria=livros associa um nome a um valor.

O que acontece com o fragmento #avaliacoes de uma URL?

A. Vai no cabeçalho Host

B. Vai no corpo da requisição

C. Não é enviado ao servidor

D. Define a porta do serviço

// quiz · 09/15 · resposta

O que acontece com o fragmento #avaliacoes de uma URL?

A Vai no cabeçalho Host

B Vai no corpo da requisição

✓ Não é enviado ao servidor

D Define a porta do serviço

// por quê

O fragmento é usado pelo próprio navegador e não integra a requisição.

O que significa receber uma resposta 404 Not Found?

A. O servidor está desligado

B. O servidor respondeu: recurso não encontrado

C. Houve falha no TLS

D. Sucesso sem corpo

// quiz · 10/15 · resposta

O que significa receber uma resposta 404 Not Found?

A. O servidor está desligado

✓ O servidor respondeu: recurso não encontrado

C. Houve falha no TLS

D. Sucesso sem corpo

// por quê

404 é uma resposta HTTP válida: o servidor recebeu, interpretou e não achou o recurso.

O que indica um código da família 5xx?

A Sucesso

B Redirecionamento

C Falha do servidor ao processar

D Informação

// quiz · 11/15 · resposta

O que indica um código da família 5xx?

A Sucesso

B Redirecionamento

✓ Falha do servidor ao processar

D Informação

// por quê

5xx aponta o servidor; 4xx aponta problemas na solicitação.

Se o servidor estiver desligado, qual código HTTP o navegador recebe?

A 404

B 500

C Nenhum: é falha de conexão

D 200

// quiz · 12/15 · resposta

Se o servidor estiver desligado, qual código HTTP o navegador recebe?

A 404

B 500

✓ Nenhum: é falha de conexão

D 200

// por quê

Sem servidor, não existe resposta HTTP. O navegador mostra um erro de conexão, como ERR_CONNECTION_REFUSED.

O que o HTTPS protege durante o transporte?

A. Endereços IP e portas

B. Método, caminho, cabeçalhos e corpo

C. O horário da comunicação

D. A existência da comunicação

// quiz · 13/15 · resposta

O que o HTTPS protege durante o transporte?

A. Endereços IP e portas

✓ Método, caminho, cabeçalhos e corpo

C. O horário da comunicação

D. A existência da comunicação

// por quê

O conteúdo HTTP é criptografado; IPs, portas, horários e volume continuam observáveis.

Qual método costuma enviar dados de um formulário para processamento?

A GET

B HEAD

C POST

D OPTIONS

// quiz · 14/15 · resposta

Qual método costuma enviar dados de um formulário para processamento?

A GET

B HEAD

✓ POST

D OPTIONS

// por quê

POST envia dados, geralmente no corpo da requisição.

No Chrome DevTools, onde vemos o corpo enviado em uma requisição POST?

A Payload

B Timing

C Elements

D Console

// quiz · 15/15 · resposta

No Chrome DevTools, onde vemos o corpo enviado em uma requisição POST?

- ✓ Payload
- B Timing
- C Elements
- D Console

// por quê

Na aba Network, a seção Payload mostra o corpo enviado; Response mostra o que voltou.